DressApp Complete Technical User Manual

Comprehensive user manual and technical reference guide for the DressApp personal wardrobe ecosystem, styling engine, circular marketplace, and administration panels.

1. Overview & Technology Stack

DressApp is an AI-driven personal wardrobe manager, styling advisor, and circular marketplace. It helps users manage clothing items digitally, auto-crop and tag them, receive weather- and calendar-aware outfit recommendations, scan EU Digital Product Passports (DPP), and trade garments.

Core Value Proposition

- **Digital Wardrobe Ingestion**: Snapshot or upload photo processing with automated background removal, clothing categorization, and attribute tag generation.
- **AI Virtual Stylist**: A conversational agent that contextually reviews your wardrobe, Google Calendar events, and local weather forecasts to suggest daily outfits.
- **Circular Marketplace**: Secure peer-to-peer buying, selling, swapping, and renting of clothes to reduce fast fashion waste.
- **Cost-Per-Wear (CPW) Analytics**: Insights into wardrobe capitalization value, utilization rates, and usage optimization.

Technology Architecture

- **Backend Edge**: Python 3.11 with FastAPI, using asynchronous Motor drivers connected to a MongoDB Atlas cluster.
- **Frontend SPA**: React 19 single-page application utilizing `useSyncExternalStore` custom stores (`stylistStore`, `dailySuggestionsStore`, `useOutfitStore`, `useClosetStore`, `useSuitcaseStore`), Tailwind CSS, Shadcn/UI primitives, and `react-i18next` supporting 12 locales.
- **State & Network Optimization**: In-flight request deduplication, 15-minute store caching, and `visibilitychange` tab revalidation yielding zero background GET requests when idle.
- **Local Machine Learning & Sizing**: CPU-local U2-Net (`rembg`) background matting, SegFormer-b2 clothing parsing, Fashion-CLIP embeddings, and ANSUR II physical body measurement regression model (`body_predictor.py`). Optionally routes to self-hosted GPU containers (SegFormer-b3 + BiRefNet) for fast operations.
- **Conversational STT/TTS**: Real-time client-side Web Speech recognition fallback, multimodal server-side Gemini 2.5 Flash modulations, and on-device offline Piper/Sherpa-ONNX engines.
- **External Integration Services**: OpenWeatherMap API for weather fetching, Google Calendar OAuth for daily schedule exports, OpenStreetMap (Nominatim) address autocomplete, and PayPal Subscriptions/Checkout REST APIs.

2. Prerequisites

Host Environment Requirements

- **Hardware**: Minimum 4 GB RAM VPS (e.g., Hetzner VPS hosting the production `dressapp.co`).
- **Dependencies**: Docker & Docker Compose stack (including backend, frontend, and Caddy TLS termination).
- **Environment Variables**: API keys configuration (`GEMINI_API_KEY`, `DEEPGRAM_API_KEY`, `OPENWEATHER_API_KEY`, `PAYPAL_LIVE_CLIENT_ID/SECRET`, and Google Calendar OAuth tokens).

User App Requirements

- **Web Browser**: Google Chrome or Apple Safari (required for full voice feature compatibility).
- **Permissions**: Grant Camera permission (for clothing snapshots and QR scans) and Microphone permission (for voice conversation).
- **Network**: Active connection for LLM processing, with IndexedDB caching providing offline catalog browsing.

3. Step-by-Step Instructions

3.1 Ingesting Garments (Adding Items)

INGESTION PARADIGMS: Photography, EU Product Passports, and Digital Commerce Receipts.

A. Interactive Camera and File Upload

1. Navigate to the **Add Item** screen.
2. Select **Take Photo** (launches native mobile camera) or click **Upload Photos** (opens OS file chooser).
3. The client computes the image's SHA-256 and horizontal difference-hash (dHash) in-browser (~100-180ms) to check against your existing closet.
4. If a match is found, the **Duplicate Preflight Dialog** opens showing matching previews. Select **Skip** or **Add anyway**.
5. Once accepted, the server starts an NDJSON stream. A placeholders preview frame displays within 5-7 seconds, allowing you to edit the item details immediately while the backend completes tagging.
6. Verify auto-detected tags (color, fabric, fit, pattern, occasion). If the cutout shape is incorrect, change the **Category** dropdown; this triggers SegFormer to automatically re-crop the garment.
7. Click **Save** to optimistically paint the item in the closet grid immediately (~16ms) while background WebP thumbnail generation concludes.

B. Scanning EU Digital Product Passports (DPP)

1. Tap the **Scan QR (DPP)** button on the Add Item page.
2. Grant camera permissions and align the QR code printed on the garment's retail label, or upload a saved QR screenshot.
3. The backend resolves the URL and executes SSRF safety checks (blocking private IP ranges).
4. The system scrapes the JSON-LD schemas to extract brand, materials composition, supply-chain trace, carbon footprint, and care guidelines.
5. Review the extracted data displayed in the green **Verified DPP Data** accordion panel and click **Save**.

C. Importing Digital Commerce Receipts

1. Open the **Digital Import** tab.
2. Choose a sub-mode: **Paste Text**, **Upload Image**, **Upload PDF**, or input a **Web Link**.
3. The backend uses multimodal vision models to extract transaction facts (brand, price, size, category).
4. Parsed fields are receipt-locked to protect them from future visual re-analysis. Click **Save** to confirm.

3.2 Conversational AI Virtual Stylist

Describe styling dilemmas and receive hands-free spoken outfit advice.

1. Navigate to the **AI Stylist** screen.
2. Click the microphone icon `[Microphone]` in the chat input bar.
3. Speak your request (e.g., "What top matches my beige trousers for a rainy outdoor lunch?").
4. If Web Speech is supported, your voice transcribes live in the input box. If not, the app records a WebM file and uploads it.
5. The backend routes the voice query to the local Gemma4 container (falling back to Gemini 2.5 Flash transcription if offline).
6. The stylist processes your wardrobe history, local weather forecasts, and calendar events to formulate a styling proposal.
7. The stylist speaks the response using preselected voice profiles (`puck`, `aoede`, or `charon`).
8. Tap **Play reply** (or **Replay** in Hebrew mode) on the card to replay the voice audio.

3.3 Profile, Preferences, and Subsystem Dependencies

The Profile page serves as the core control panel for DressApp. Configuration fields directly impact the performance, routing, and behavior of downstream modules.

Accordion Section Dependencies & Rationale

1. **Photos & Digital Avatar Stage (`AvatarViewer2D` & `DynamicAvatar`)**

- **Why does it matter?**: Renders your visual identity across all try-on canvases using a dual-mode stage (segmented real-body photo cutout vs dynamic 2D Bezier vector SVG mannequin).

- **Subsystem Dependencies**: Photo cutouts are background-matted via local U2-Net (`rembg`) and downscaled in-browser to a maximum of 1280px at 82% quality to fit within MongoDB's 16MB document ceiling. The stage applies calibrated positional landmarks (`top-[14.5%]` collar-to-neckline, `top-[36.5%]` waistband-to-waistline, `bottom-[2%]` footwear plane) and proportional chest/hip scaling ($scaleX$). Click *Remove Photo* to instantly switch back to the 2D SVG vector mannequin.

2. Style Profile (Modest rules, Dress code)

- **Why does it matter?**: It establishes personal boundaries for recommended outfits, preventing the AI from generating inappropriate style suggestions.

- **Subsystem Dependencies**: The selected parameters (e.g., Modest clothing constraints) are fed directly into the styling prompts for Gemini 2.5 Flash, filtering matching wardrobe outputs before they are shown.

3. Details (Name, Phone, Occupation)

- **Why does it matter?**: It customizes the communication tone and routes notification alerts.

- **Subsystem Dependencies**: The user's name is dynamically parsed into emails and system-level pushes. The phone number serves as a fallback registry for scheduled alerts. The occupation parameter is fed to the stylist LLM and Trend Scout personalization ranker to customize proposals.

4. Body Measurements & Sizing (ANSUR II Regression Model)

- **Why does it matter?**: It eliminates sizing guesswork, allowing external retail size comparison and accurate virtual layering.

- **Subsystem Dependencies**: Entering 4 basic parameters (**Height**, **Weight**, **Waist**, **Foot Length**) triggers the scikit-learn ANSUR II regression model (`body_predictor.py`) to auto-predict 6 structural dimensions (*Shoulders*, *Chest*, *Hip*, *Sleeve*, *Inseam*, *Outseam*). Measurements are queried directly by the **Shopping Assistant** Chrome Extension content scripts to read size tables on partner websites (Zara, Asos) and recommend sizes.

5. Lifestyle (Status, Sex)

- **Why does it matter?**: It tailors default recommendations and scores content algorithms.

- **Subsystem Dependencies**: The sex selection directly affects the ranking logic of daily Trend Scout cards. If a news card category mismatches the user's sex, the algorithm applies a -2.0 score penalty, demoting it in the feed.

6. AI Configuration (SaaS keys, edge mode, credits)

- **Why does it matter?**: It determines billing routing, operational performance, and network offline status.

- **Subsystem Dependencies**: Routes text/audio generation queries. Standard setups consume DressApp system credits. Inputting personal API keys (Google AI Studio, Anthropic, OpenAI) redirects charges to the user's developer billing accounts. Selecting edge local mode routes queries to the offline Gemma container.

7. Scheduler & Push (Frequency, daily alarm, style focus)

- **Why does it matter?**: It manages automatic daily style pushes.

- **Subsystem Dependencies**: Activates `APScheduler` cron jobs on the FastAPI backend. Every morning, it triggers push notifications via `pywebpush` using the client's VAPID keys, matching the selected style focus parameters.

8. Google Calendar (OAuth sync, export rules)

- **Why does it matter?**: It bridges your wardrobe directly with your real-world calendar events.

- **Subsystem Dependencies**: Authenticates via Google OAuth. The scheduler queries your calendar to identify events, formats outfits, and pushes calendar events straight to your Google Calendar agenda.

9. Location Services (GPS tracking, weather accuracy)

- **Why does it matter?**: It coordinates weather-appropriate suggestions and local transaction radius filters.

- **Subsystem Dependencies**: Triggers `navigator.geolocation` reverse-geocoding. Coordinates are sent to OpenWeatherMap API to adjust the stylist recommendations (e.g., rainwear for downpours). It also calculates distances for local Marketplace listings and experts (e.g.,Lisbon radius checks).

10. Voice & Language (Virtual stylist voice selection)

- **Why does it matter?**: It establishes text dictionary locales and voice modulations.

- **Subsystem Dependencies**: Controls the active language for translations via `react-i18next`. The voice selection maps BCP-47 voice codes (e.g., `he-IL` or `ar-JO`) to client Web Speech synthesis voices or offline Piper TTS models.

11. Invite Friends (Share payload API)

- **Why does it matter?**: It provides a viral loop for free closet expansion.

- **Subsystem Dependencies**: Appends the referrer's MongoDB ID to the URL. New registrations dynamically query this ID and atomically increment the referrer's `closet_capacity_bonus` by +10 slots, modifying the limit guards in `closet.py`.

3.4 Wardrobe Insights Dashboard

Analyze wardrobe capitalization value, tracking garment utilization, and cost-per-wear parameters.

1. Navigate to **Wardrobe Insights**.
2. **Review Metrics**:

- *Closet Worth*: Dynamic summation of purchase prices.

- *Closet Utilization*: Percentage of closet garments worn at least once.

- *Average Cost-per-Wear (CPW)*: Computed as `Price / Wear Count`.

3. **Distribution Graphs**: Toggle tabs to view Recharts visualizations:

- *Color Palette*: Mapped hex codes distribution.

- *Materials*: Fabric percentages distribution.

- *Subcategories*: Mapped subcategories.

4. **Efficiency Leaderboard**: View the top 5 garments showing the lowest Cost-per-Wear scores.

3.5 Outfit Canvas & Planner

Build, layer, and review outfit proposals on an interactive 2D avatar canvas.

1. Open the **Outfit Canvas** planner.
2. **Outerwear Layering (Dual Canvas)**: If your outfit includes outerwear (e.g., a jacket) over a top, the page renders two vertical canvas modules: "With Outerwear" (showing the jacket layered) and "Without Outerwear" (revealing the top underneath).
3. **Interactive 2D Elements**: Tap directly on any clothing piece on the avatar's body. The app routes you straight to that garment's detail screen.
4. **Review Metrics Tab**: Click the details button and choose the **Metrics** tab to view progress bars for compatibility criteria:

- *Color Harmony* (neutral harmony)

- *Pattern Compatibility* (pattern clashing prevention)

- *Body Fitting* (size matching)

- *Weather Match* (season suitability)

- *Event Match* (activity appropriateness)

- *Location Match* (modest rules checks)

5. **Rename/Describe**: Click the Pencil icon to edit outfit names and descriptions.

3.6 Suitcase Assistant

Organize your packing requirements for trips without overpacking.

1. Go to the **Suitcase** page and fill out the Trip Context form (destination, start/end dates, trip category, calendar events).
2. The AI generates a customized packing list and daily outfits based on trip duration and weather forecasts.
3. Review the packing progress. If an important item is missing (e.g., umbrella for rain, swimwear for beach), the system alerts you and suggests matches from the marketplace or local stores.
4. Use the integrated chat box to refine suggestions (e.g., "Change day 2 to casual evening wear"). The assistant edits the suitcase while preserving the rest of the list.
5. Tap **Approve Suitcase** to finalize your plan.

3.7 Scheduler & Push Reminders

Set daily styling alerts to receive outfit recommendations automatically.

1. Open **Profile** and go to **Scheduler & Push**.
2. Toggle notifications, set a daily notification time, weekday frequency, and styling focus theme.
3. Every morning, the background cron task (`APScheduler`) checks weather forecasts and sends a push notification.
4. Tap the notification on your device (or view the web app's Notification Center) to open a proposal dialog displaying 3 styled suggestions.
5. Save a suggestion directly to your **Wardrobe Diary**.

3.8 Marketplace (Resale, Renting, Swapping, Gifting)

Engage in the peer-to-peer circular fashion marketplace.

• **Create a Listing**: Open an item's detail page, select **Edit Intent**, and choose a non-private intent:

- *For Sale*: Input list price and currency (detects your default currency via regional locale preferences).

- *Rent*: Set daily rental tariff and borrowing conditions.

- *Swap*: Mark item open for trade.

- *Donate*: Publish item for free.

• **State Synchronization**: Listings propagate to the feed automatically. The client uses `useSyncExternalStore` and IndexedDB caching to load search parameters with zero-latency.

• **Try-On Sandbox**: Renters/buyers can test-fit a listing against items in their private closet before checking out.

• **Transactional Checkout**:

- *Purchasing/Renting*: Complete transaction via integrated PayPal buttons. Captured webhooks notify the seller, flip listing state to sold/rented, and log transactions in the ledger minus the 7% platform fee.

- *Bartering (Swapping)*: Prospective swappers propose trades. Lister receives confirmation emails to accept or decline.

3.9 Admin Panel Dashboard

System liveness validation, financial bookkeeping, and user account management.

1. Navigate to `/admin` (available to administrator roles).
2. **Overview**: Audit Gross Volumes and Platform Fee income summaries. Inspect the **Provider Activity Table** to view downstream liveness statistics (Gemini API, weather service latency, and error rates).
3. **Providers**: Click **Verify Key** to send a direct ping to the Gemini API. Toggle the **Eyes Vision Override** switch to route image analysis between the default Gemini endpoint and a local Gemma container.
4. **Users**: View active credits, roles, and lifetime payments. Use direct actions to Promote or Demote users.
5. **Listings**: View listing states and toggle active flags to suspend fraudulent items.

---

4. Expected Results

• **Ingestion**: Items immediately populate the closet grid (~16ms). Background cutouts render clean, transparent PNG results.
• **DPP Verified Badge**: Scanning valid passports displays the green information card with sustainability details.
• **Avatar Outerwear**: Outerwear displays correctly layered over tops on the 2D avatar canvas without clipping headwear/shoes.
• **Voice Response**: Virtual Stylist text outputs play spoken audio automatically with a visible waveform indicator.
• **Subscriptions**: Activating Pro immediately removes the 150-item limit warning.

---

5. Troubleshooting

HTTP 402 Payment Required

• **Problem**: Ingestion blocked. You have reached the maximum baseline limit of 150 closet items.
• **Solution**: Go to Profile -> Subscription and upgrade to Pro, or share your invite link to get +10 slots per registration.

SSRF Blocked / DNS Error on DPP

- **Problem**: Scanned QR passport URL fails to parse.
- **Solution**: The parser blocks private IP addresses (e.g. `127.0.0.1`, `192.168.x.x`) to protect internal servers. Ensure QR codes point to public domains.

Camera / Microphone Permission Denied

- **Problem**: Capture/scan viewport displays an 'X' error screen, or voice typing fails.
- **Solution**: Open browser permissions, enable Camera and Microphone access for the domain, and reload.

Stylist Chat Failure / Rate Limits

- **Problem**: Chat shows errors or freezes.
- **Solution**: The server catches Gemini `429` rate limits and falls back to a rule-based closet selection algorithm. Verify your internet connection.

Out of Memory (OOM) VPS Spikes

- **Problem**: CPU/RAM peaks during upload processes.
- **Solution**: Ingestion utilizes sequential queue locks for batches >5 items. Ensure the server has at least 4 GB RAM.

---

6. Limitations

- **Browser Web Speech APIs**: Native Speech-to-Text translation is restricted to Chrome and Safari; other browsers fall back to standard text input.
- **Offline Client Modulations**: Mobile offline Piper ONNX speech synthesis uses fewer voice profiles than the server-side Gemini audio modal.
- **Image Size Constraints**: Avatar and profile uploads are compressed locally in the browser to 82% quality to fit within the 16MB MongoDB document boundary.
- **Receipt Parsing Scope**: Highly blurred, distorted, or handwritten receipts may fail data extraction.